Preuve de concept

Optimisation de trajectoire par B-Spline Initialement pour le Robot Club Toulon

SeaTech, Toulon, France

Décembre 2025 - Octobre 2026

Timm Claverie



Table des matières

1 Introduction **1**

2 Phase 1 : Génération du Dataset et Modélisation **2**
2.1 Fondements Théoriques : B-Splines et Courbes de Bézier 2
2.1.1 Définition et Distinction 2
2.1.2 Équation Canonique 2
2.2 Couplage Géométrie-Cinématique 2
2.2.1 Démonstration 3
2.2.2 Calcul du point de contrôle P_2 (Arrivée) 3
2.2.3 Implémentation Algorithmique 4
2.3 Le Solveur Temporel (Time Optimal Solver) 4

3 Phase 2 : Apprentissage Supervisé **6**
3.1 Architecture du Modèle 6
3.2 Normalisation et Reconstruction 6
3.3 Stratégie Hybride IA + Physique 7

4 Extension au cas 2D et Validation Temps Réel **8**
4.1 Génération du Dataset : Couverture 360° et Solveur Exact 8
4.2 Apprentissage Supervisé : Un Réseau Neuronal Ultra-Léger 9
4.3 Contrôleur Hybride et Exécution 50 Hz 9
4.4 Résultats Visuels et Analyse Cinématique 10
4.5 Conclusion du POC 11

5 Extension au cas 3D **12**
5.1 Génération du Dataset : Repère Local 3D 12
5.2 Apprentissage Supervisé 12
5.3 Contrôleur Hybride et Exécution 50 Hz 12
5.4 Résultats Visuels et Analyse Cinématique 13
5.5 Conclusion du POC 3D 14

6 Système de trajectographie 3D pour drone **15**

7 Conclusion **16**

Résumé

Ce rapport présente une chaîne hybride de trajectographie pour robots mobiles, combinant courbes de Bézier cubiques, apprentissage supervisé et raffinement cinématique. Un dataset de trajectoires optimales est d'abord généré hors-ligne sous contraintes de vitesse et d'accélération, puis un réseau de neurones ultra-léger est entraîné pour prédire le temps optimal de parcours. L'inférence est sécurisée par un ajustement analytique garantissant le respect des limites $V_{\max}$ et $A_{\max}$. L'approche, validée en 2D puis étendue en 3D, permet une exécution temps réel compatible avec une boucle à 50 Hz. Initialement destiné à un robot footballer au sol, le système est appliqué à un drone en vol et ouvre la voie à des missions embarquées à vitesse d'arrivée contrôlée.

Mots clés : *trajectographie*, *courbes de Bézier*, *apprentissage supervisé*, *contrôle cinématique*, *temps réel*, *drone*, *robot mobile*

1 Introduction

Ce rapport présente la conception d'un contrôleur de trajectoire pour robots mobiles combinant apprentissage supervisé et contraintes cinématiques. L'objectif est de remplacer les solveurs de planification lourds par une chaîne hybride légère : un réseau de neurones estime le temps optimal T_{opt} d'un déplacement entre deux points, puis un raffinement analytique garantit le respect strict des vitesses et accélérations maximales $V_{\max}$ et $A_{\max}$.

La démarche s'articule en trois étapes : génération hors-ligne d'un dataset de trajectoires de Bézier cubiques contraintes par les vitesses de départ et d'arrivée, entraînement d'un réseau ultra-léger, puis inférence embarquée avec correction physique. L'architecture est d'abord validée en 2D, avant d'être étendue en 3D via une base locale orthonormée.



Initialement destiné à un robot footballer au sol pour le Robot Club Toulon (RCT), le système est ensuite adapté à un drone évoluant en volume 3D. Les résultats montrent une exécution compatible avec une boucle à 50 Hz, ouvrant la voie à des applications embarquées telles que l'évitement d'obstacles ou l'interception à vitesse contrôlée.

2 Phase 1 : Génération du Dataset et Modélisation

Cette première phase est critique : elle constitue la "Vérité Terrain" (*Ground Truth*) nécessaire à l'apprentissage supervisé. Le script `generateur.py` utilise une approche inverse : il détermine analytiquement la trajectoire géométrique idéale pour satisfaire des contraintes cinématiques temporelles.

2.1 Fondements Théoriques : B-Splines et Courbes de Bézier

2.1.1 Définition et Distinction

Dans la planification de trajectoire, nous cherchons à définir une courbe paramétrique continue $C(u)$. Deux familles mathématiques sont couramment utilisées : les B-Splines et les courbes de Bézier.

— **B-Splines (Basis Splines) :** Ce sont des fonctions définies par morceaux, offrant un contrôle local. Modifier un point de contrôle n'affecte qu'une partie de la courbe. Elles sont idéales pour des trajectoires longues et complexes traversant de multiples points de passage (waypoints).

— **Courbes de Bézier :** C'est un cas particulier de B-Splines. Une courbe de Bézier de degré n est définie par $n+1$ points de contrôle. Contrairement aux B-Splines générales, une courbe de Bézier passe **toujours** par son premier (P_0) et son dernier point (P_n), ce qui est impératif pour notre problème (départ et arrivée précis du robot).

Dans le cadre de ce projet, le script utilise des **Courbes de Bézier Cubiques** (degré 3). Ce choix offre le compromis optimal : assez de degrés de liberté pour définir la position et la tangence (vitesse) aux deux extrémités, sans la complexité de calcul des B-Splines de degré supérieur.

2.1.2 Équation Canonique

Une courbe de Bézier cubique est définie pour un paramètre adimensionnel $u \in [0, 1]$ par :

$$B(u) = (1-u)^3 P_0 + 3(1-u)^2 u P_1 + 3(1-u)u^2 P_2 + u^3 P_3 \tag{2.1}$$

Où :

— P_0 : Position de départ (Robot).

— P_3 : Position d'arrivée (Cible).

— P_1, P_2 : Points de contrôle influençant la tangente et la courbure.

2.2 Couplage Géométrie-Cinématique

Le calcul des points P_1 et P_2, habituellement fixés arbitrairement, sont ici calculés pour respecter scrupuleusement les vitesses v_{start} et v_{end}.

2.2.1 Démonstration

Pour comprendre les formules utilisées dans le code, il faut dériver l'équation de Bézier par rapport au temps t. Soit T la durée totale du mouvement. On a la relation $u = \frac{t}{T}$, donc $\frac{du}{dt} = \frac{1}{T}$.

La vitesse vectorielle $\vec{V}(t)$ est la dérivée temporelle de la position :

$$\vec{V}(t) = \frac{dB(u)}{du} \cdot \frac{du}{dt} = \frac{dB(u)}{du} \cdot \frac{1}{T} \tag{2.2}$$

Calculons la dérivée par rapport à u à l'origine ($u = 0$, soit le départ) :

$$\begin{aligned} \frac{dB}{du}(0) &= \lim_{u \to 0} \frac{d}{du} \left[(1-u)^3 P_0 + 3(1-u)^2 u P_1 + \dots \right] \\ &= 3(P_1 - P_0) \end{aligned}$$

En réinjectant dans l'équation de vitesse au temps $t = 0$:

$$\vec{V}_{start} = 3(P_1 - P_0) \cdot \frac{1}{T} \tag{2.3}$$

Si l'on isole P_1, on obtient la formule utilisée dans le code :

$$P_1 = P_0 + \frac{\vec{V}_{start} \cdot T}{3} \tag{2.4}$$

En notant $\vec{V}_{start} = v_{start} \cdot \vec{u}_{start}$ (norme $\times$ direction), on retrouve exactement l'implémentation Python. Le facteur 3 provient donc directement du degré du polynôme cubique.

2.2.2 Calcul du point de contrôle P_2 (Arrivée)

Le point P_2 joue un rôle symétrique à P_1 : il conditionne l'approche finale vers la cible P_3. Il permet de définir l'angle d'arrivée (tangente) et la vitesse finale instantanée (v_{end}).

Dérivation Mathématique

Tout comme pour le départ, nous partons de la dérivée de la courbe de Bézier par rapport au paramètre u, mais cette fois-ci évaluée à la fin de la trajectoire, c'est-à-dire en $u = 1$.

La dérivée d'une courbe de Bézier cubique en $u = 1$ est déterminée par le vecteur formé par les deux derniers points de contrôle :

$$\frac{dB}{du}(1) = 3(P_3 - P_2) \tag{2.5}$$

En appliquant la règle de la chaîne pour passer du domaine géométrique (u) au domaine temporel (t), avec $\vec{V}(t) = \frac{dB}{du} \cdot \frac{1}{T}$, nous obtenons la vitesse à l'instant final $t = T$:

$$\vec{V}_{end} = \frac{1}{T} \cdot 3(P_3 - P_2) \tag{2.6}$$

Notre objectif est d'isoler P_2. En manipulant l'équation :

$$\begin{aligned} T \cdot \vec{V}_{end} &= 3(P_3 - P_2) \\ \frac{T \cdot \vec{V}_{end}}{3} &= P_3 - P_2 \\ P_2 &= P_3 - \frac{T \cdot \vec{V}_{end}}{3} \end{aligned}$$

Interprétation Géométrique et Code

L'équation finale révèle une différence fondamentale avec P_1 : la présence du signe **moins**.

Géométriquement, cela s'explique simplement : le vecteur vitesse $\vec{V}_{end}$ pointe vers "l'avant" (la direction du mouvement à l'arrivée). Or, le point P_2 est situé *avant* P_3 dans le temps. Pour trouver P_2 à partir de la cible P_3, il faut donc "reculer" le long de la tangente.

Dans le script `generateur.py`, cette logique est implémentée explicitement :

```
# u_end : Vecteur unitaire de direction a l'arrivee
u_end = np.array([np.cos(theta_end), np.sin(theta_end)])

# Distance de recul proportionnelle a la vitesse et au temps
dist_tan_end = (v_end * T) / 3.0

# On part de la cible (P3) et on soustrait le vecteur
P2 = P3 - dist_tan_end * u_end
```

Listing 2.1 – Calcul "Rétrograde" de P2

Cette formulation garantit que si le robot doit arriver avec une vitesse élevée (v_{end} grand), P_2 sera placé loin en arrière de P_3, forçant la trajectoire à s'aligner progressivement bien avant l'arrivée. À l'inverse, si $v_{end} = 0$ (arrêt au stand), alors $P_2 = P_3$, ce qui géométriquement écrase la fin de la courbe pour forcer une décélération.

2.2.3 Implémentation Algorithmique

Le code traduit cette contrainte mathématique dans la fonction `get_control_points_from_velocity` :

```
def get_control_points_from_velocity(P0, P3, theta_start, theta_end, v_start, v_end, T):
    # Vecteurs unitaires de direction
    u_start = np.array([np.cos(theta_start), np.sin(theta_start)])
    u_end   = np.array([np.cos(theta_end), np.sin(theta_end)])

    # Application de la formule derivee ci-dessus
    # V(0) = 3(P1-P0)/T  =>  dist = v * T / 3
    dist_tan_start = (v_start * T) / 3.0
    dist_tan_end   = (v_end * T) / 3.0

    P1 = P0 + dist_tan_start * u_start
    P2 = P3 - dist_tan_end * u_end

    return P1, P2
```

Listing 2.2 – Calcul des points de contrôle sous contrainte

2.3 Le Solveur Temporel (Time Optimal Solver)

Une fois la géométrie liée au temps T par les formules ci-dessus, il reste une inconnue : **Quel est le temps T minimal réalisable ?**

Si T est trop court, la distance $P_0 \to P_1$ devient très petite (car proportionnelle à T), mais la vitesse demandée reste la même, ce qui implique une accélération mathématiquement nécessaire qui peut dépasser les capacités du moteur.

Le script utilise une approche de **Recherche Dichotomique** (bisection) pour trouver ce T :

1. Tolérance de 5 ms pour trouver le T minimal faisable
2. Pour chaque candidat T_{test} :
   — On calcule P_1 et P_2 avec les formules de vitesse.
   — On échantillonne la courbe (25 points).
   — On calcule l'accélération en tout point : $\vec{A}(t) = \frac{d^2B}{dt^2}$.
3. Si $\max(||\vec{A}(t)||) \leq A_{max_robot}$ et $\max(||\vec{V}(t)||) \leq V_{max_robot}$, alors T_{test} est valide.
4. Le premier T valide trouvé est conservé comme T_{opt}.

C'est cette valeur T_{opt}, issue d'une simulation physique complexe, qui servira d'étiquette (Label) pour entraîner le réseau de neurones dans la phase suivante.

3 Phase 2 : Apprentissage Supervisé

Le script `entrainement.py` charge le dataset généré pour entraîner un modèle de régression léger.

3.1 Architecture du Modèle

Le réseau de neurones, nommé `TimePredictorNet`, est un Perceptron Multi-Couches (MLP) conçu pour l'inférence rapide. Le modèle $6 \to 18 \to 5 \to 1$ (environ 250 paramètres). Cette légèreté permet une inférence en 0.07 ms.

- **Entrée (6 neurones) :** $x_{local}, y_{local}, \cos\theta_{target}, \sin\theta_{target}, v_{start}, v_{end}$.
- **Couches Cachées :**
  - Linear(18) + ReLU
  - Linear(5) + ReLU
- **Sortie (1 neurone) :** Temps Optimal T.

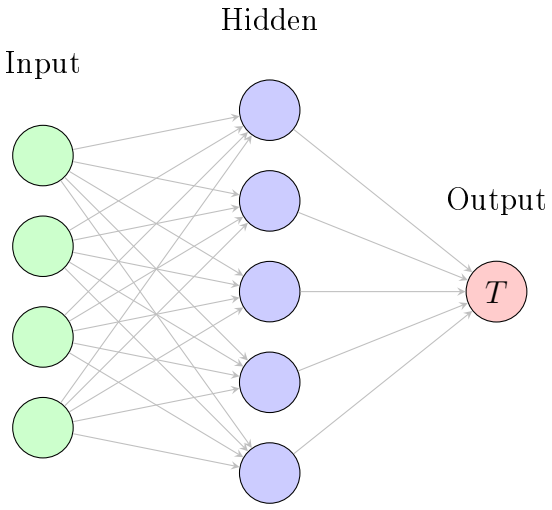


Figure 3.1 – Exemple de schéma simplifié d'une architecture neuronale

3.2 Normalisation et Reconstruction

Un point critique relevé dans le code est la normalisation des données (Z-Score normalization). Les statistiques (*mean*, *std*) sont sauvegardées dans `statistique_de_normalisation.npz` car elles sont indispensables pour l'inférence.

Le modèle ne prédit que le temps. La géométrie de la courbe est reconstruite géométriquement après la prédiction, garantissant ainsi que la trajectoire reste mathématiquement une courbe de Bézier valide, même si la prédiction du temps comporte une légère erreur.

3.3 Stratégie Hybride IA + Physique

L'IA fournit une estimation initiale très rapide ("Initial Guess"). Cependant, pour garantir la sécurité du robot, cette estimation est raffinée par un algorithme d'optimisation local.

Le processus se déroule comme suit :

1. **Prédiction :** Le modèle estime T_{pred}.
2. **Audit :** On calcule les pics d'accélération et de vitesse pour ce T_{pred}.
3. **Correction :** Si les contraintes sont violées (ratio > 1), on augmente T. Si le robot est en sous-régime, on diminue T.

La prédiction du réseau est raffinée par une bisection stricte sur les contraintes physiques (marge de sécurité de 1% sur Vmax et Amax). Cela garantit mathématiquement une trajectoire réalisable, sans dépassement, en un temps borné (maximum 40 itérations).

4 Extension au cas 2D et Validation Temps Réel

Ce chapitre présente l'évolution du système vers un POC complet pour la robotique 2D. L'objectif est de générer des trajectoires en 360°, de garantir le respect strict des contraintes cinématiques, et de valider une exécution à 50 Hz sur CPU.

4.1 Génération du Dataset : Couverture 360° et Solveur Exact

La version précédente générait uniquement des cibles situées devant le robot (angle restreint). Pour un POC robuste, le dataset a été régénéré avec une couverture totale de 360° (cibles devant et derrière le robot).

Le solveur temporel a également été amélioré. Au lieu d'un balayage linéaire, le script `generateur.py` utilise une **recherche par dichotomie** pour trouver le T_{opt} quasi-optimal sous tolérance respectant $V_{max} = 6.5$ m/s et $A_{max} = 6.0$ m/s^2.

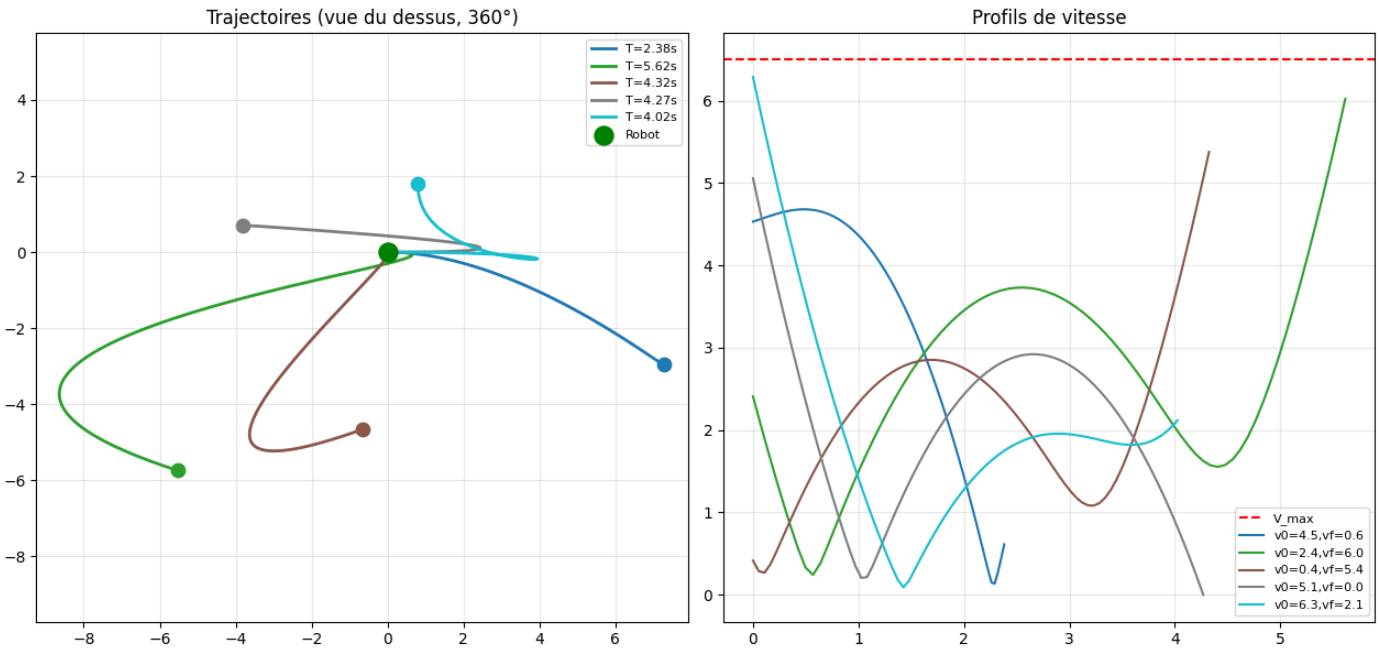


Figure 4.1 – Trajectoires générées en 360° et profils de vitesse associés. On observe que les cas nécessitant un demi-tour génèrent des trajectoires plus longues et des temps T plus élevés, respectant scrupuleusement les limites physiques.

— **Dataset final** : 20 000 échantillons.

— **Temps optimal** (T_{opt}) : varie de 0.8 s à 8.5 s selon la distance et le profil de vitesse.

4.2 Apprentissage Supervisé : Un Réseau Neuronal Ultra-Léger

Pour garantir l'exécution sur un processeur standard (CPU) sans GPU, le réseau `TimePredictorNet` a été réduit à sa plus simple expression tout en conservant sa capacité de prédiction. Il ne prédit qu'une seule valeur : le temps optimal T_{opt}.

— **Entrées (6)** : x_{local}, y_{local}, $\cos(\theta)$, $\sin(\theta)$, v_{start}, v_{end}.

— **Couches** : `Linear(6, 18)` → ReLU → `Linear(18, 5)` → ReLU → `Linear(5, 1)`.

— **Paramètres** : ≈ 250 poids.

L'entraînement (80% Train / 20% Validation) a été effectué sur 300 époques avec un scheduler réduisant le learning rate si la validation stagne.

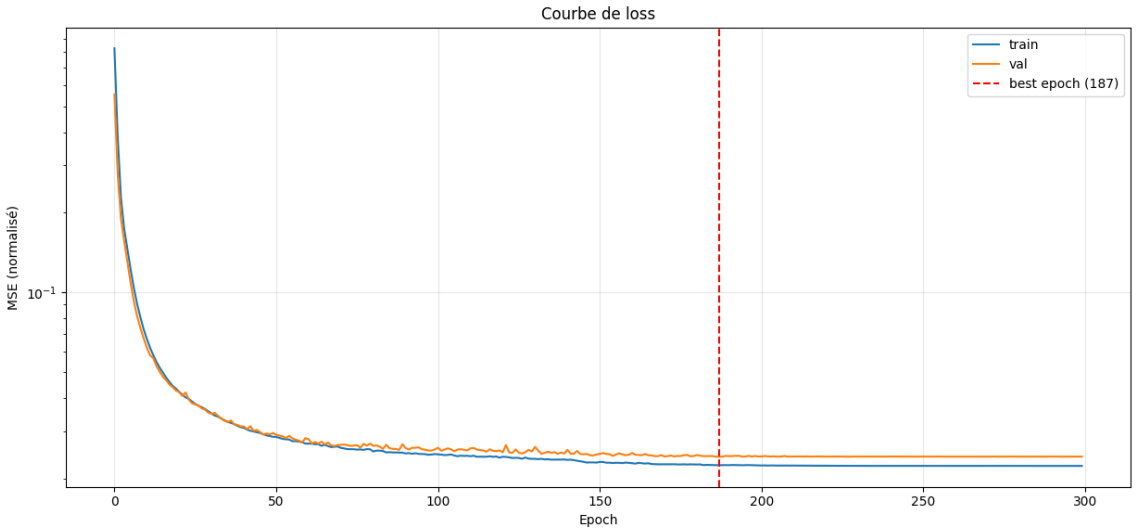


FIGURE 4.2 – Courbe d'apprentissage. Le modèle converge rapidement dès l'époque 50, et le meilleur modèle (*Early Stopping*) est atteint à l'époque 187. L'écart train/val est minime, prouvant l'absence de surapprentissage.

4.3 Contrôleur Hybride et Exécution 50 Hz

L'inférence pure du réseau neuronal (0.07 ms) est couplée à un module physique `refine_time_with_physics`. Ce module utilise une **dichotomie** pour s'assurer que les points de contrôle Bézier respectent à 99% les limites du robot (marge de sécurité de 1%).

Pour valider le POC, un exécuteur (`TrajectoryExecutor`) a été implémenté avec deux modes :

1. **Mode CACHED** : Le plan est calculé une fois, on ne fait qu'échantillonner la courbe.
2. **Mode REPLAN** : On recalcule le plan à chaque pas de temps (simulation de cibles mouvantes ou d'évitement d'obstacles dynamique).

```
======================================================================
 BENCHMARK 50 Hz — pipeline Bézier-cubique contraint
 Budget : 20 ms / pas
======================================================================
[INIT] Modèle chargé : modele_temps.pth
[INIT] Stats chargées.
[MODE] modèle entraîné chargé

=== BENCHMARK PAR BRIQUE ===
  ☑ NN forward seul                      p50= 0.070 ms  p95= 0.076 ms  p99= 0.093 ms
  ☑ compute_plan (NN + physique)         p50= 3.826 ms  p95= 3.933 ms  p99= 4.267 ms
  ☑ sample_plan (évaluation pure)        p50= 0.018 ms  p95= 0.018 ms  p99= 0.021 ms
  ☑ step replan (plan + sample)          p50= 3.896 ms  p95= 4.004 ms  p99= 4.140 ms
  ☑ step cached (sample seul)            p50= 0.017 ms  p95= 0.018 ms  p99= 0.021 ms

=== DÉMO 50 Hz — MODE CACHED (exécution) ===
  Cible         : T_opt = 2.99 s → 150 pas
  Latence       : p50=0.018 ms | p99=0.025 ms | max=0.030 ms
  Fréquence tenue : ☑ OK

=== DÉMO 50 Hz — MODE REPLAN (replanification à chaud) ===
```

FIGURE 4.3 – Benchmark de latence. Le mode "cached" s'exécute en 0.018 ms. Le mode "replanification" complet (NN + Physique + Bézier) s'exécute en p99=4.14 ms. Les deux modes respectent largement le budget de 20 ms par pas pour une boucle à 50 Hz.

4.4 Résultats Visuels et Analyse Cinématique

Le scénario de test le plus critique est celui où la cible est située **derrière** le robot, avec un changement d'orientation (ex : robot en $(0, 0)$ vers la cible en $(-2, -1)$). Ce cas force le robot à effectuer un virage très serré.

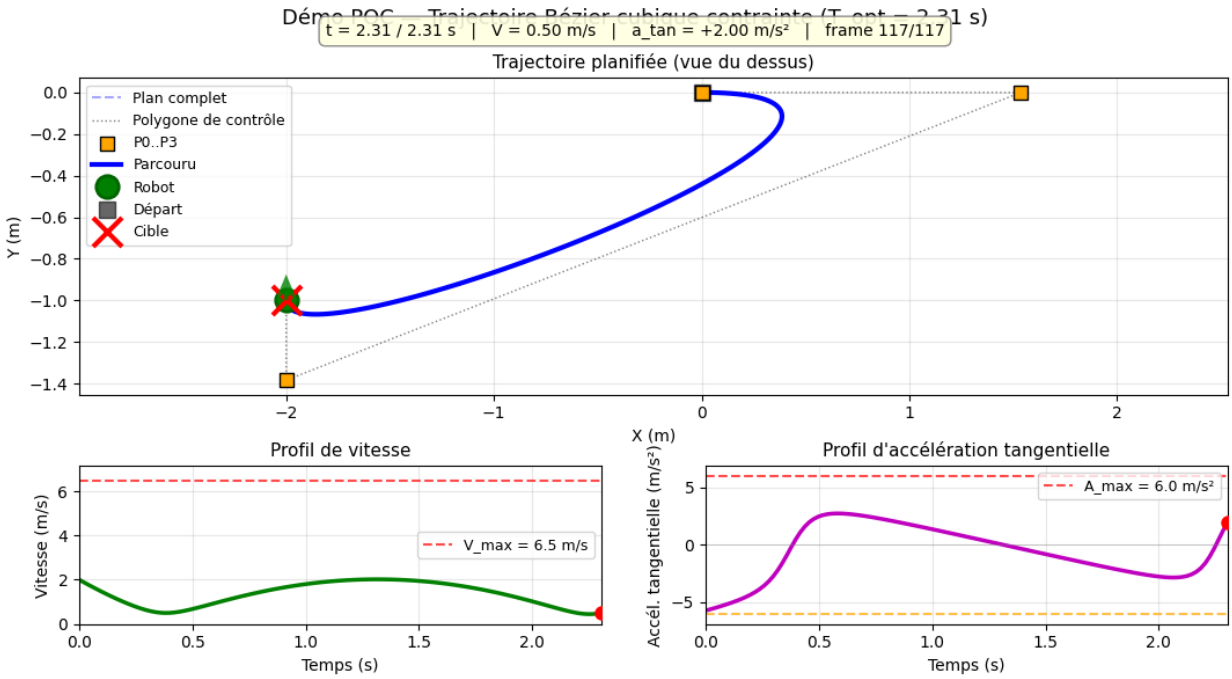


FIGURE 4.4 – Démo de la trajectoire vers $(-2, -1)$. Le robot ralentit fortement au milieu de la courbe pour respecter la force centrifuge. À l'arrivée, l'accélération tangentielle atteint exactement la limite de freinage (-6.0 m/s^2), validant le respect strict des contraintes physiques.

- **Analyse de la trajectoire** : Le profil de vitesse montre une décélération de 2.0 m/s à 0.5 m/s pour passer le virage, puis une ré-accélération.
- **Respect des contraintes** : Le profil d'accélération tangentielle sature exactement à $+6.0$ et -6.0 m/s^2, prouvant que le solveur physique extrait le maximum des capacités du robot (*time-optimal*).
- **Embarqué** : Le réseau de neurones fournit le T_{opt} en une fraction de milliseconde, permettant de recalculer cette trajectoire complexe 50 fois par seconde si nécessaire.

4.5 Conclusion du POC

Ce POC démontre qu'il est possible de remplacer un solveur d'optimisation non-linéaire lourd par un réseau de neurones de 250 paramètres, tout en garantissant la sécurité cinématique via un raffinement analytique. L'architecture est prête pour être déployée sur un microcontrôleur ou un Raspberry Pi pour du contrôle temps réel.

5 Extension au cas 3D

Ce chapitre présente l'adaptation du système pour la navigation en volume 3D. L'architecture globale (réseau ultra-léger, reconstruction analytique et raffinement physique) reste identique à la version 2D. Les différences résident dans la gestion du repère local et dans les dimensions des tenseurs d'entrée.

5.1 Génération du Dataset : Repère Local 3D

La difficulté majeure du passage en 3D est le changement de repère. En 2D, une simple rotation d'angle θ suffisait. En 3D, le robot évolue dans un espace où sa direction est un vecteur tridimensionnel.

Pour que le réseau de neurones puisse apprendre efficacement, le problème doit être invariant par rotation. Nous construisons donc une **base orthonormée locale** $(\vec{u}, \vec{v}, \vec{w})$ attachée au robot :

- $\vec{u}$: Vecteur directeur du robot (normalisé).
- $\vec{v}$: Vecteur orthogonal à $\vec{u}$ (généré par produit vectoriel avec l'axe Z global, ou l'axe Y si $\vec{u}$ est colinéaire à Z).
- $\vec{w}$: Vecteur orthogonal complétant la base ($\vec{w} = \vec{u} \wedge \vec{v}$).

La matrice de passage R (de taille 3×3) permet de projeter la cible dans le repère local du robot :

$$\vec{P}_{local} = R \cdot (\vec{P}_{cible} - \vec{P}_{robot})$$

La cible est ainsi générée aléatoirement dans une sphère complète (coordonnées sphériques) et son vecteur d'arrivée est également projeté. Le dataset résultant contient 8 entrées au lieu de 6 : $(x_{loc}, y_{loc}, z_{loc}, u_x, u_y, u_z, v_{start}, v_{end})$.

Le solveur temporel (dichotomie) reste inchangé, mais les normes de vitesse et d'accélération sont calculées en 3D : $\|\vec{V}\| = \sqrt{v_x^2 + v_y^2 + v_z^2}$.

5.2 Apprentissage Supervisé

L'architecture du réseau `TimePredictorNet` a été légèrement élargie pour absorber la dimension supplémentaire :

- **Entrées (8)** : x_{loc}, y_{loc}, z_{loc}, u_x, u_y, u_z, v_{start}, v_{end}.
- **Couches** : `Linear(8, 20)` $\rightarrow$ ReLU $\rightarrow$ `Linear(20, 10)` $\rightarrow$ ReLU $\rightarrow$ `Linear(10, 1)`.
- **Paramètres** : ≈ 400 poids.

La cible d'apprentissage reste le temps optimal T_{opt}. La reconstruction des points de contrôle de Bézier s'effectue ensuite de manière déterministe dans le repère local, avant d'être ramenée dans le repère global via la transposée de la matrice R.

5.3 Contrôleur Hybride et Exécution 50 Hz

Le contrôleur physique `refine_time_with_physics` vérifie les contraintes sur la courbe 3D. Pour valider les performances, le benchmark mesure les mêmes briques d'exécution.

```
======================================================================
 BENCHMARK 50 Hz — pipeline Bézier-cubique 3D
 Budget : 20 ms / pas
======================================================================
[MODE] modèle entraîné 3D chargé

=== BENCHMARK PAR BRIQUE (3D) ===
  ☑ NN forward seul (3D)                  p50= 0.138 ms  p95= 0.240 ms  p99= 0.282 ms
  ☑ compute_plan (NN + physique 3D)       p50= 3.077 ms  p95= 5.842 ms  p99= 6.847 ms
  ☑ sample_plan (évaluation pure 3D)      p50= 0.018 ms  p95= 0.019 ms  p99= 0.037 ms
  ☑ step replan (plan + sample 3D)        p50= 3.051 ms  p95= 3.234 ms  p99= 5.602 ms
  ☑ step cached (sample seul 3D)          p50= 0.018 ms  p95= 0.019 ms  p99= 0.032 ms

=== DÉMO 50 Hz — MODE CACHED (exécution 3D) ===
  Cible        : T_opt = 2.36 s -> 118 pas
  Latence      : p50=0.019 ms | p99=0.023 ms | max=0.037 ms
  Fréquence tenue : ☑ OK

=== DÉMO 50 Hz — MODE REPLAN (replanification à chaud 3D) ===
  1000 pas de replanification
  Latence      : p50=5.441 ms | p95=5.986 ms | p99=8.530 ms | max=10.946 ms
  Fréquence tenue : ☑ OK

======================================================================
 ☑ POC 3D VALIDÉ — le pipeline tient 50 Hz dans les deux modes
======================================================================
>>>
```

FIGURE 5.1 – Benchmark de latence pour le pipeline 3D. Le calcul complet d'un plan (NN + Physique) s'exécute en $p99 = 6.8$ ms. Le mode "replanification à chaud" (simulant un mouvement continu du robot) prend au maximum 10.9 ms par cycle. Les deux modes respectent largement le budget de 20 ms pour une boucle à 50 Hz.

5.4 Résultats Visuels et Analyse Cinématique

Ce scénario de test illustre un déplacement avec un fort changement d'élévation. Le robot part de l'origine vers une cible en hauteur, nécessitant une courbure 3D.

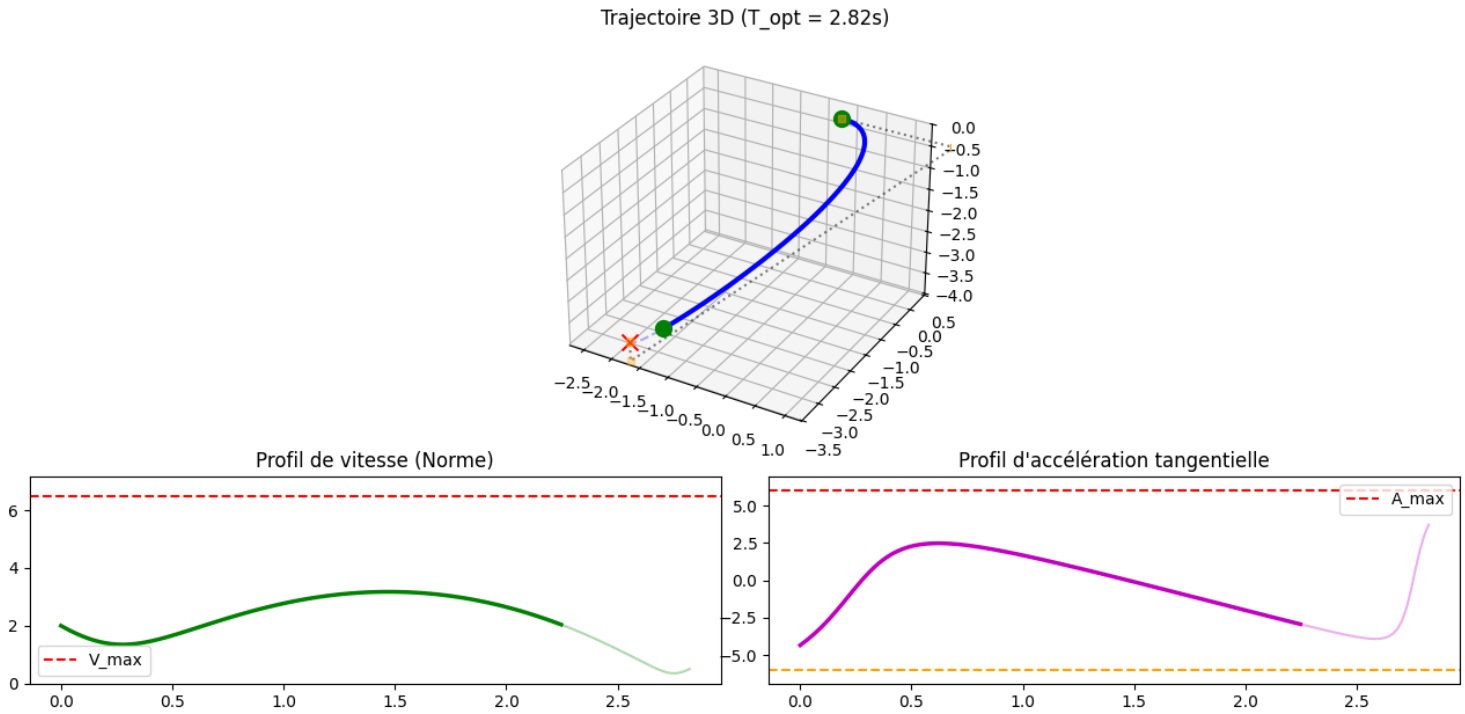


FIGURE 5.2 – Démonstration de la trajectoire 3D. En haut : la trajectoire dans le volume (courbe bleue). En bas : les profils de vitesse et d'accélération tangentielle. Les limites V_{max} et A_{max} sont parfaitement respectées, démontrant l'efficacité du raffinement physique dans l'espace 3D.

— **Analyse de la trajectoire** : La norme de la vitesse (graphique en bas à gauche) montre une accélération franche pour sortir de l'axe initial, suivie d'une phase de croisière. Le profil d'accélération tangentielle sature à $+6.0$ et -6.0 m/s^2.

— **Embarqué** : La légèreté du réseau (400 poids) permet d'effectuer cette planification 3D en moins de 7 ms, ouvrant la voie à des applications de drones ou de bras manipulateurs temps réel.

5.5 Conclusion du POC 3D

L'extension en 3D démontre la robustesse de l'architecture hybride. Le passage d'un problème 2D à un problème 3D ne nécessite qu'une adaptation de la base locale (matrice de rotation) et une légère modification du réseau. Le cœur du système (recherche de T_{opt} et reconstruction de Bézier) reste identique et garantit une exécution temps réel sans explosion combinatoire.

6 Système de trajectographie 3D pour drone

Ce chapitre illustre, au travers d'une simulation, un cas où un drone en vol reçoit des consignes de déplacement dans un environnement 3D. On suppose que le système de décision du drone fournit à notre modèle de trajectoire des waypoints exprimés dans le repère local du drone, de la forme $[x, y, z, dir_x, dir_y, dir_z, vitesse]$: (x, y, z) désigne la position relative de la cible, (dir_x, dir_y, dir_z) la direction d'arrivée souhaitée, et $vitesse$ la vitesse de passage au waypoint.

Travailler dans un repère relatif présente deux intérêts majeurs. D'une part, le calcul du point visé ne dépend plus d'un repère absolu : le modèle est donc plus robuste aux dérives de localisation, et la nécessité d'un SLAM est supprimée. D'autre part, la consigne transmise au modèle est directement exploitable par le contrôleur bas niveau du drone, sans étape de transformation intermédiaire.

Le drone ne contrôle pas l'intégralité de sa trajectoire à l'avance : si un obstacle apparaît sur la trajectoire planifiée, le recalcul complet — qui s'effectue à moins de 20 Hz — permet d'insérer de nouveaux waypoints afin de contourner l'obstacle, tout en préservant la continuité de vitesse et de cap.



Figure 6.1 – Démonstration d'une trajectoire 3D calculée par le modèle, avec un drone évoluant en forêt et évitant les arbres.

Les principaux avantages du modèle sont :

1. un calcul de trajectoire en environ 10 ms, compatible avec une replanification à 50 Hz ;
2. la possibilité d'imposer la vitesse d'arrivée au passage de chaque waypoint.

7 Conclusion

Ce rapport a présenté une chaîne complète de génération, d'apprentissage et d'exécution de trajectoires pour robots mobiles. En résumé, la démarche repose sur trois étapes : (i) la génération hors-ligne d'un dataset de trajectoires optimales à partir de courbes de Bézier cubiques contraintes par les vitesses de départ et d'arrivée, (ii) l'entraînement d'un réseau de neurones ultra-léger prédisant le temps optimal T_{opt}, et (iii) une exécution hybride où l'inférence neuronale est sécurisée par un raffinement analytique des contraintes cinématiques $V_{\max}$ et $A_{\max}$. L'architecture a d'abord été validée en 2D, puis étendue en 3D via une base locale orthonormée et une matrice de rotation. Les benchmarks montrent que le pipeline respecte largement le budget temps réel : inférence du réseau en environ 0.07 ms, calcul complet d'un plan 3D en quelques millisecondes, et replanification à chaud compatible avec une boucle à $\sim$ 50Hz, soit un budget de 20 ms par pas. La démonstration en environnement 3D, avec un drone évoluant en forêt et contournant des arbres, illustre le bon fonctionnement global du système.

L'objectif initial de ce travail était de concevoir un système de trajectographie pour un robot footballer au sol, capable de rejoindre un waypoint avec une vitesse de départ, une vitesse d'arrivée et une direction imposées, tout en minimisant le temps de parcours et en respectant les limites physiques du robot. Ce cadre a ensuite été étendu à un système 3D pour drone. Le passage en 3D ne remet pas en cause l'architecture : il nécessite principalement une adaptation du repère local, une projection de la cible dans ce repère et un élargissement léger du réseau de neurones. Le cœur du système — prédiction du temps optimal, reconstruction géométrique de la courbe de Bézier et raffinement physique — reste identique.

Enfin, la légèreté du modèle neuronal, environ 400 poids dans sa version 3D, et sa très faible latence d'inférence permettent d'envisager son embarquement sur un drone à impact, également appelé drone kamikaze ou munition rôdeuse. Dans ce contexte, le modèle peut planifier une trajectoire relative vers une cible et imposer une vitesse d'arrivée ainsi qu'une direction d'impact précises, tout en respectant les contraintes de vitesse et d'accélération maximales. La capacité de replanification à 50 Hz offre de plus une marge intéressante pour l'évitement d'obstacles ou l'adaptation à une cible mobile. Ce travail ouvre donc la voie à des perspectives d'embarquement réel, de validation sur drone physique et d'extension à des missions d'interception ou d'assaut terminal à vitesse contrôlée.